

實作專案 (五): 部署 AI 專案並打造作品集

各位同學好,

恭喜大家！我們已經走完了 Vibe Coding 的三個階段，從撰寫 PRD、與 AI 協同生成原型，到最後親手完善，你手上現在應該有一個獨一無二的專案網站了（例如：老王咖啡廳、吉他社網站...）。

現在，我們要進行最後的，也是最重要的一步：發佈這個新專案，並將它與你的個人履歷串連起來，打造一個真正的「個人作品集」！

專案目標

1. 為你的 AI 協作專案建立一個新的 GitHub 倉庫 (Repository)。
2. 將這個新專案發佈到 GitHub Pages。
3. 在你的「個人履歷」中，加入連到這個「新專案」的連結。
4. 在你的「新專案」中，加入連回「個人履歷」的連結。

部署你的 AI 協作專案

老師的真心話 (開始前請檢查！)

在開始上傳之前，請先確認你的 GitHub Desktop 已經登入！

尤其是在公用電腦，如果你忘了登入，你可能會無法「Publish」(發佈)，或是(更糟的)不小心用了前一位同學的帳號上傳。

檢查與解決方法：

1. 在 GitHub Desktop 中，點擊左上角 "File" > "Options..." (macOS 上是 "GitHub Desktop" > "Preferences...")。
2. 在 "Accounts" 分頁中，確認顯示的是你自己的 GitHub 帳號。
3. 如果不是，請先 "Sign out" (登出)，然後 "Sign in" (登入) 你自己的帳號。

這次的部署流程和「履歷專案」有一點點不同。請仔細跟著步驟操作：

步驟一：建立一個「全新」的倉庫 (Repository)

我們不要把這個 new 專案和履歷專案 (your-username.github.io) 混在同一個倉庫裡。我們要為這個新專案建立一個全新的家。

為了讓大家有最一致的學習體驗，我們將沿用「履歷專案」的上傳流程：

1. 打開 **GitHub Desktop**。
2. 點擊左上角「File」> "New Repository..." (建立新倉庫)。
3. 在跳出的視窗中，填寫以下資訊：
 - **Name:** 請為這個專案取一個簡短、有意義的英文名稱。例如：coffee-shop、guitar-club 或 my-vibe-project。(注意：這裡的名稱將會成為你網址的一部分！)
 - **Description:** (選填) 簡單描述一下這個專案。
 - **Local Path:** 確認路徑是你熟悉的資料夾 (例如 Documents/GitHub)。
 - 不要勾選 "Initialize this repository with a README"。

4. 點擊 "Create repository"。

步驟二：將本地專案上傳 (Push) 到新倉庫

1. 複製檔案：
 - 現在, GitHub Desktop 已經在你的電腦上建立了一個空的專案資料夾 (例如 Documents/GitHub/coffee-shop)。
 - 請打開你存放 AI 專案的資料夾 (裡面有 index.html、mystyles.css 等檔案)。
 - 將這些檔案全部複製, 然後貼上到剛剛 GitHub Desktop 建立的那個新資料夾中 (貼到 Documents/GitHub/coffee-shop 裡面)。
2. 提交 (Commit):
 - 回到 **GitHub Desktop**, 你會看到所有剛剛貼上的檔案都出現在 "Changes" 列表中。
 - 在左下角 **Summary** 欄位輸入「首次上傳 Vibe 專案」。
 - 點擊 "Commit to main"。
3. 發佈 (Publish):
 - 點擊主畫面上方的 "Publish repository" 按鈕。
 - 在跳出的視窗中:
 - **Name:** 保持名稱不變 (例如 coffee-shop)。
 - **!! 重要 !!** 取消勾選 "Keep this code private" (保持程式碼私有)。GitHub Pages 必須是公開的。
 - 點擊 "Publish repository"。

你的 AI 專案現在已經成功上傳到 GitHub 了！

步驟三：啟動這個新專案的 GitHub Pages

這是和履歷專案最不一樣的地方！我們需要手動開啟這個新倉庫的 Pages 功能。

1. 回到 GitHub 網站, 進入你剛剛建立的新倉庫 (例如 github.com/你的帳號/coffee-shop)。
2. 點擊上方標籤列的 "Settings" (設定)。
3. 在左側選單點擊 "Pages"。
4. 在 "Build and deployment" (建置與部署) 區塊:
 - **Source:** 選擇 "Deploy from a branch"。
 - **Branch:** 選擇 **main** (或 master) 分支, 並將資料夾保持在 /(root)。
 - 點擊 "Save"。
5. 儲存後, 頁面會重新整理。請耐心等待 1-5 分鐘。

你的新專案網址將會是：**https://你的使用者名稱.github.io/你的倉庫名稱/**

例如：<https://sung-chien-lin.github.io/coffee-shop/>

最終任務：建立作品集連結

一個好的作品集, 會讓所有專案「互連」。現在我們有兩個網站了, 該讓它們彼此認識一下。

老師的真心話 (公用電腦請注意！)

如果你是在會被清空的公用電腦上操作, 你本地的資料夾已經不在了。

在執行「任務一」和「任務二」之前，你需要先打開 **GitHub Desktop**，點擊 "File" > "**Clone Repository...**"，從你的 GitHub 帳號列表中選擇你要修改的倉庫（例如 your-username.github.io 或 coffee-shop），將它「複製 (Clone)」一份回電腦上。

接著才能打開那個資料夾進行修改、Commit 並 Push。請養成先 **Clone** 再修改的好習慣！

任務一：在「履歷」中，加入連到「新專案」的連結

1. 在你電腦中，打開你的「履歷專案」資料夾（也就是 your-username.github.io 那個）。(若使用公用電腦，請先執行上面的 Clone 步驟)。
2. 用 VS Code 打開 index.html。
3. 找到你的「作品集 (Portfolio)」或「專案 (Projects)」區塊 (如果你當初有建立的話)。
4. 新增一個連結，指向你剛剛發佈的「新專案」網址。
 - 例如，在你的履歷中新增一段 HTML：

```
<div>
  <h3>老王咖啡廳形象網站 (AI 協作專案)</h3>
  <p>使用 PRD 與 Vibe Coding 流程，與 Gemini 協同開發的靜態網站。</p>
  <a href="[https://你的使用者名稱.github.io/coffee-shop/]"(https://你的使用者名稱.github.io/coffee-shop/)" target="_blank">點此觀看網站</a>
</div>
```
5. 儲存 index.html。
6. 打開 **GitHub Desktop**，它會自動切換到你的履歷倉庫 (如果沒有，請在左上角手動切換)。
7. Commit 並 Push 你的履歷更新。

任務二：在「新專案」中，加入連回「履歷」的連結

1. 在你電腦中，打開你的「AI 協作專案」資料夾 (例如 coffee-shop)。(若使用公用電腦，請先執行上面的 Clone 步驟)。
2. 用 VS Code 打開 index.html。
3. 找到頁尾 <footer> 區塊。
4. 加入一個連結，指回你的「個人履歷」網址。
 - 例如，在 <footer> 中新增一行：

```
<p class="text-center mt-4">
  網站由 <a href="[https://你的使用者名稱.github.io/]"(https://你的使用者名稱.github.io/)" target="_blank">你的名字</a> 製作
</p>
```
5. 儲存 index.html。
6. 打開 **GitHub Desktop**，切換到你的新專案倉庫 (例如 coffee-shop)。
7. Commit 並 Push 這次的更新。

總結

恭喜你！你現在不只擁有一個個人首頁，還擁有了一個獨立的專案網站，並且成功地將它們連結

起來，構成了一個具備基本雛形的「個人作品集」。

這套「本地修改 -> GitHub Desktop Commit & Push -> 網站自動更新」的流程，就是現代網頁開發的基礎。你已經掌握了最重要的技能！